

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Módulos y paquetes

Como hemos visto, un módulo o paquete no es otra cosa que una colección de código, clases, variables y funciones que podemos utilizar desde nuestros programas. Lo importamos al principio del programa y a partir de ahí podremos utilizarlo en nuestro código.

Por **módulo** entendemos un fichero que incluye código y un **paquete** será un conjunto de módulos que se suelen incluir en un directorio.

```
paquete
├── modulo1.py
├── modulo2.py
└── modulo3.py
```

Y dentro de cada fichero moduloX.py tendríamos funciones, clases, ...

Existen diferentes formas de importación según queramos importar todo el paquete o solamente unos módulos del mismo.

Con “*import paquete*” lo importamos completo. Para usarlo usaremos sus clases, funciones o variables precedidas del nombre del módulo, al que llamaremos *espacio de nombres* (namespace):

```
import os

ficheros = os.listdir('/tmp')
os.rename('fichero1', 'fichero2')
```

En nuestro “paquete” imaginario sería:

```
import paquete

paquete.modulo1.funcion1()
```

El código dentro del módulo se agrupa según el espacio de nombres, por ejemplo *os.path.isfile()*.

Si queremos importar sólo alguna parte del código podemos usar la estructura de “*from ... import clase*”, pudiendo evitar la referencia al módulo. Por ejemplo:

```
from os import path
path.isfile('test')
```

Para agilizar el uso del código de los módulos podemos definir un alias con la palabra reservada “as” en cualquier de los formatos (*import* o *from ... import*)

```
import os.path as p
p.isfile('test')
```

En nuestro ejemplo, si sólo queremos usar el módulo2 haríamos

```
from paquete import modulo2
```

...

¿Dónde se buscan los módulos?

Cuando importamos un paquete ¿dónde se busca este paquete?:

1. En primer lugar se busca en la librería estándar de python, es decir entre todos los paquetes que incluye la versión que estamos usando
2. Después se busca en todos los directorios incluidos en “sys.path”, donde se incluyen en este orden:
 - a. El directorio actual donde estamos ejecutando nuestro código
 - b. Los directorios de instalación de python y algunos otros según un orden concreto.

Podemos ver lo que incluye nuestro `sys.path` con este sencillo código:

```
>>> import sys
>>> for p in sys.path:
...     print(p)
...
```

```
/usr/lib/python36.zip
/usr/lib/python3.6
/usr/lib/python3.6/lib-dynload
/home/javacasm/.local/lib/python3.6/site-packages
/home/javacasm/Dropbox/Cursos/Drones/DJITelloPy
/usr/local/lib/python3.6/dist-packages
/usr/lib/python3/dist-packages
```

El estar ordenado de esta manera hace que podamos incluir un paquete que sobrescriba otro. Por ejemplo poniendo un paquete “os” en el directorio actual, al hacer “import os” se importaría el nuestro.

Importando nuestro código

Cuando creamos un programa cuyo código está distribuido entre varios ficheros, podemos importar la funcionalidad como si un módulo se tratara, haciendo simplemente `import fichero` (sin la extensión .py)

Accederemos a la funcionalidad con `fichero.funcion` del mismo modo.

Puedes probarlo importando el código del fichero de las 20 preguntas y usándolo desde tu propio programa.

Cuando hacemos esto tenemos que distinguir si estamos usando el código como un módulo o como un programa principal.

En la siguiente sección vamos a ver cómo hacerlo.

Propiedades de los módulos

Cuando ejecutamos un intérprete Python existen una [serie de variables](#) con valores características, como por ejemplo:

- **name** que se refiere al nombre completo del módulo en ejecución. Cuando ejecutamos un fichero, el valor de este atributo será **'main'**, cuando importamos un módulo será el nombre del módulo.

Podemos aprovechar esto para saber si nuestro código se está ejecutando directamente o a través de una importación.

```
if __name__ == '__main__':
    myMain()
else:
    print('Me han importado')
```

- **version** nos dice la versión de un módulo usando

```
import os
os.__version__
```

- **file** nombre del fichero

Vamos a probarlo. Creamos un fichero *Prop_modulos.py* con este contenido:

```
'''
Módulo de prueba
'''

try:
    print(f'__name__: {__name__}')
    print(f'__file__: {__file__}')
    print(f'__loader__: {__loader__}')
    print(f'__package__: {__package__}')
    print(f'__path__: {__path__}')
except Exception as e:
    print(f'Error: {e}')
```

Si lo ejecutamos con *python3 Prop_modulos* obtenemos:

```
__name__: __main__
__file__: Prop_modulos.py
__loader__: <_frozen_importlib_external.SourceFileLoader object at
0x7fcb9d9d02b0>
__package__: None
Error: name '__path__' is not defined
```

En cambio si desde un intérprete python3 hacemos *import Prop_modulos* veremos:

```
>>> import Prop_modulos
__name__: Prop_modulos
__file__: /home/javacasm/Dropbox/Cursos/CursoPython/codigo/Prop_modulos.py
__loader__: <_frozen_importlib_external.SourceFileLoader object at
0x7fd7a67433c8>
__package__:
Error: name '__path__' is not defined
```

Si ahora hacemos *help(Prop_modulos)* veremos que nos muestra parte de esta información.

Librería Estándar

Actualmente Python incorpora una enorme librería por defecto, es la llamada Librería Estándar y podemos ver todos los detalles en su [documentación](#)

Ya conocemos algunos de estos módulos como *os*, *time* o *random*. Vamos a ver una descripción de los principales módulos:

- **os** todo lo relacionado con el sistema operativo (en general): ficheros, hilos, variables del entorno, procesos, permisos, usuarios, redes,
- **sys** temas específicos al sistema operativo donde se está ejecutando: propiedades de los ejecutables, de la configuración del sistema,...
- **csv** para trabajar en un nivel básico con archivos de datos de tipo csv
- **math** todo lo relacionado con las matemáticas: funciones, constantes,...
- **glob**: para trabajar con comodines al listar ficheros.
- **datetime**: fechas y horas
- **statistics**: para realizar operaciones estadísticas.
- **zip**: para trabajar con ficheros comprimidos zip.

- **time**: medida del tiempo y esperas.
- **Multimedia**: para reproducir y generar ficheros multimedia
- **Protocolos de Internet**: para utilizar los protocolos más frecuentes de internet.
- **random**: números y elecciones aleatorios

Ya hemos trabajado algunos de ellos y ahora vamos a ver ejemplos de otros.

Módulo sys

El módulo **sys** tiene las utilidades propias del sistema operativo donde estamos ejecutando el intérprete:

- `sys.argv`: argumentos con los que hemos ejecutado el programa
- `sys.executable`: ejecutable del intérprete actual.
- `sys.exit()`: sale del programa python actual.
- `sys.float_info`: información sobre las características del tipo decimal.
- `sys.getsizeof(objeto)`: nos dice la memoria que ocupa el objeto indicado.
- `sys.maxsize`: máximo entero que podemos usar.
- `sys.modules`: diccionario con los módulos cargados.
- `sys.version`: versión del intérprete que estamos usando.

Módulo math

En el módulo **math** como no podía ser de otra forma nos vamos a encontrar todo tipo de funciones matemáticas:

- Funciones trigonométricas, como *sin*, *cos*, *tan*,... y sus inversas *asin*, *acos*, *atan*, ...
- Raíces cuadradas con *sqrt*
- Exponenciales con *exp* y logaritmos con *log*
- Constantes como *math.pi*, *math.e*, *math.inf* (infinito)

Como ejemplo vamos a ver las diferentes formas de redondear un número decimal a entero

Empezamos por la función *round()* que produce el resultado al que estamos acostumbrados:

- Los números con parte decimal igual o menor de 0.5 se redondean al entero anterior

- Si la parte decimal es mayor que 0.5 se redondea al entero siguiente:

```
>>> numeros = (0.5, 0.51, 1.6, 2.5, 2.51)
>>> print('Round')
>>> for numero in numeros:
...     print(f'{numero} -> {round(numero)}')
Round
0.5 -> 0
0.51 -> 1
1.6 -> 2
2.5 -> 2
2.51 -> 3
```

El módulo *math* incluye otras 2 formas de redondear:

- *math.floor()* que redondea siempre al entero anterior

```
>>> import math
>>> print('math.floor')
>>> for numero in numeros:
...     print(f'{numero} -> {math.floor(numero)}')
math.floor
0.5 -> 0
0.51 -> 0
1.6 -> 1
2.5 -> 2
2.51 -> 2
```

- *math.ceil()* redondea al entero superior ``

```
>>> import math
>>> print('math.ceil')
>>> for numero in numeros:
...     print(f'{numero} -> {math.ceil(numero)}')
math.ceil
0.5 -> 1
0.51 -> 1
1.6 -> 2
2.5 -> 3
2.51 -> 3
```

Trabajando con fracciones

Aunque no forman parte del módulo *math*, sino que tienen su propio módulo, en la biblioteca estándar de Python podemos encontrar las clases necesarias para trabajar con *fracciones*. El módulo no podía tener otro nombre que **fractions**, donde trabajaremos con la clase **Fraction**, definiendo su *numerador* y su *divisor*:

```
>>> from fractions import Fraction
>>> f1 = Fraction(1,2)
>>> f2 = Fraction(1,3)
```

Por supuesto podemos imprimirlas:

```
>>> print(f1)
1/2
>>> print(f2)
1/3
```

Vemos que podemos operar entre ellas si recurrir a un valor decimal, lo que las hace más exactas:

```
>>> f3 = f1 + f2
>>> f3
Fraction(5, 6)
```

Y también podemos hacer operaciones entre fracciones y números “normales”:

```
>>> f4 = 3 * f2
>>> f4
Fraction(1, 1)
>>> 1 == f4
True
```

y que en todo momento se simplifican los resultados:

```
>>> f5 = Fraction(3,24)
>>> f5
Fraction(1, 8)
```

Además el módulo tiene otros métodos, como por ejemplo **gcd** que nos permite calcular el máximo común divisor:

```
>>> import fractions
>>> fractions.gcd(15,5)
5
```

Ficheros comprimidos

Existen varios módulos para trabajar con [tareas propias de compresión y descompresión](#): `zlib`, `gzip`, `bz2`, `lzma`, `tarfile` y `zipfile`.

Vamos a ver un ejemplo de cómo trabajar con un fichero zip estándar. El uso es similar al tratamiento de los ficheros normales:

- Lo abrimos indicando si lo queremos para lectura “r” o para escritura “w”
- ```
from zipfile import ZipFile
```

```
ficheroZip = ZipFile('ficheroZIP.zip', mode='r') # Lo abrimos en modo lectura
"r"
```



```
print(ficheroZip.namelist()) # mostramos los ficheros que incluyen
ficheroZip.extractall() # extraemos todos los ficheros
```

```
ficheroZip.close() # Lo cerramos
```

Si lo que queremos es crear un fichero zip con varios ficheros incluidos haremos lo siguiente:

```
from zipfile import ZipFile
```

```
ficheroZip = ZipFile('nuevoFicheroZIP.zip', mode='r') # Lo abrimos en modo
escritura "w"
```

```
ficheroZip.write('fichero1.txt') # añadimos los ficheros
ficheroZip.write('fichero2.txt')
ficheroZip.write('fichero3.txt')
```

```
ficheroZip.close() # Lo cerramos
```

## Aleatoriedad con random

Podemos usar el módulo `random` cuando necesitamos algo relacionado con lo aleatorio

.

Tenemos varias posibilidades dependiendo de lo que necesitemos: \*

`randint(minimo,maximo)`: Nos da un número entero comprendido entre el valor mínimo y máximo incluyendo los dos extremos:

```
from random import randint
Recordar que se incluye el máximo
minimo <= randint(minimo,maximo) <= maximo
for i in range(10):
 print(randint(0,10))
```

- `choice(coleccion)` : Devuelve uno de los elementos de la colección que le hemos pasado:

```
from random import choice
colores = ('rojo', 'verde', 'azul', 'blanco', 'negro', 'amarillo')
for i in range(5):
 print (choice(colores))
```

- `sample(coleccion, N)`: Elige N elementos distintos de la colección que la hemos pasado y los devuelve como una tupla

```
from random import sample
colores = ('rojo', 'verde', 'azul', 'blanco', 'negro', 'amarillo')
print(sample(colores,4))
```

- `shuffle(lista)`: desordena aleatoriamente la lista que le pasamos.

Ejemplo: Generador de password

```
chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.,-:;<>][_+ '
longitud = int(input('¿longitud de la contraseña? '))
for i in range(longitud):
 passwd+=random.choice(chars)
print(passwd)
```

A veces, en contra de lo que pudiera parecer, queremos que se reproduzca una misma secuencia de números aleatorios (para pruebas, para cierta predictibilidad,...). En ese caso podemos hacer usar el método `seed(semilla)` que hará que toda la secuencia se repita si usamos la misma *semilla*:

```
>>> from random import seed, randint
>>> seed(666)
>>> for i in range(5):
... print(randint(0,10))
7
6
6
4
8
>>> seed(666)
>>> for i in range(5):
... print(randint(0,10))
7
6
6
4
8
```

## Midiendo tiempos

El módulo `time` está pensado para medir tiempos.

```
>>> import time
>>> print(time.time()) # segundos desde 1/1/1970
1664454320.520648
>>> print(time.localtime()) # estructura con los datos de fecha y hora
time.struct_time(tm_year=2022, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=25,
tm_sec=49, tm_wday=3, tm_yday=272, tm_isdst=1)
>>> print(time.strftime('%Y-%m-%d_%H:%M:%S',time.localtime()))
2022-09-29_14:28:06
```

- `time()`: Nos da un número decimal donde la parte entera son los segundos desde el 1 de enero de 1970 y la decimal la fracción de segundo. Este formato se conoce como **timestamp**

- `localtime()` Nos da una estructura con la fecha y hora descompuesta
- `strftime(formato)`: nos devuelve una cadena con el dato de fecha y hora con el formato que le indicamos
- `sleep(segundos)`: detiene la ejecución del programa durante los segundos indicados. Este valor puede ser decimal.

También podemos usar `time()` para ver el tiempo que ha pasado entre dos momentos:

```
>>> inicio = time.time()
>>> final = time.time()
>>> print(f'Han pasado {final-inicio} segundos')
Han pasado 8.950223922729492 segundos
```

## Operando con fechas con datetime

El módulo `datetime` está pensado para operar con fechas.

- `datetime(año,mes,día, hora, minuto, segundo)` crea un objeto con fecha (podemos omitir los que valores de hora, minuto y segundo).
- `datetime.year` o `datetime.month`, ... nos da los valores esperados.
- `datetime.now()` nos da la fecha actual
- `datetime.strftime(formato)` convierte la fecha a una cadena con el formato indicado:

```
>>> from datetime import datetime
>>> datetime.now().strftime('%Y-%m-%d_%H:%M:%S')
'2021-03-25_20:52:38'
```

- Podemos convertir un valor de tipo `time` (timestamp) a `datetime` con:

```
>>> datetime.fromtimestamp(time.time())
datetime.datetime(2022, 9, 29, 14, 31, 43, 116377)
```

- Podemos restar fechas, obteniendo una estructura `timedelta` de resultado, de donde podemos extraer los días con `days`, los segundos con `seconds` y los microsegundos con `microseconds`.

### Código

```
>>> from datetime import datetime
>>> nacimiento = datetime(1970,12,25,15,0,0) # Fecha y hora de nacimiento
>>> hoy = datetime.now()
>>> diferencia = hoy - nacimiento
>>> print(f'han pasado {diferencia}')
han pasado 18905 days, 23:09:40.741071
>>> print(f'tienes {diferencia.days} días, y {diferencia.seconds} segundos')
tienes 18905 días, y 83380 segundos
>>> horas = diferencia.seconds // 3600 # segundos que tiene 1 hora
```

```
>>> restoSegundos = diferencia.seconds - horas * 3600
>>> minutos = restoSegundos // 60
>>> segundos = restoSegundos - minutos * 60
>>> print(f'o {diferencia.days} días, {horas} horas, {minutos} minutos y {segundos} segundos')
o 18905 días, 23 horas, 9 minutos y 40 segundos
```

- Para obtener el día actual podemos usar `today()`

```
from datetime import date
hoy = date.today()
print(hoy)
para formatearlo a Mes día, año
print(hoy.strftime('%B %d, %Y'))
```

```
para saber el día de la semana
print(hoy.weekday())
```

Vemos que nos muestra el número de día de la semana.

También podemos trabajar con los nombres de los días de la semana o de los meses, para lo que usaremos el módulo **calendar**, que dispone de varias listas con nombres:

- **month\_name** con los nombres de los meses
- **month\_abbr** con las abreviaturas de los nombres de los meses
- **day\_name** con los nombres de los días
- **day\_abbr** con las abreviaturas de los nombres de los días

La lista de los meses tiene un elemento vacío en la primera posición (posición 0) para que se correspondan los nombres con los números habituales (enero - 1, ....), con lo que podemos sacar una lista de los nombres de los meses con:

```
>>> import calendar
>>> calendar.month_name[1:]
['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
```

O la de los días de la semana con:

```
>>> import calendar
>>> calendar.day_name[:]
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

También incluye constantes de los días de la semana y algunos meses (no sé por qué on están todos...) que corresponde con su número, por ejemplo `calendar.February` vale 2 o `calendar.TUESDAY` vale 1 y `calendar.MONDAY` vale 0

Usando esto podemos recuperar el nombre del día de la semana:

```
y para saber el nombre del día usamos calendar
>>> import calendar
>>> from datetime import date
>>> hoy = date.today()
>>> print(calendar.day_name[hoy.weekday()])
monday
```

Para obtener los nombres de los meses y días en el idioma local debemos establecer la localización con el módulo *locale*. Por ejemplo para establecer el “español” de “España” haríamos

```
>>> import locale
>>> locale.setlocale(locale.LC_ALL, 'es_ES.UTF-8')
'es_ES.UTF-8'
```

A partir de este momento este intérprete mostrará los resultados traducidos:

```
>>> import calendar
>>> calendar.month_name[1:]
['enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto',
'septiembre', 'octubre', 'noviembre', 'diciembre']
>>> calendar.day_name[:]
['lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado', 'domingo']
```

## Recuperando fechas de texto

A veces queremos recuperar una fecha de un texto, es decir, tenemos una cadena que sabemos que incluye información de fecha y de hora y queremos recuperar un *datetime* para poder utilizarlo de una forma más sencilla.

Para ello utilizaremos el método **parse** de la clase *datetime.parser* que nos permite recuperar tanto la fecha desde una cadena que sabemos que contiene una fecha, como desde un texto, por ejemplo una línea de un mensaje de error de un programa. A partir de ahí también podemos recuperar una fecha concreta.

Si tenemos seguridad de que la cadena incluye sólo una fecha lo usaremos así:

```
>>> from dateutil.parser import parse
>>> fechaStr = '10-5-2021 18:45:12'
>>> parse(fechaStr)
datetime.datetime(2021, 10, 5, 18, 45, 12)
```

En el caso de la fecha esté incluida dentro de un texto podemos usar el parámetro *fuzzy=True* que buscará la parte de la fecha y nos la devolverá:

```
>>> from dateutil.parser import parse
>>> log_line = 'INFO 2014-07-03T23:27:51 supybot Shutdown complete.'
```

```
>>> parse(log_line, fuzzy=True)
datetime.datetime(2014, 7, 3, 23, 27, 51)
```

También podemos instalar otro paquete que nos permite parsear fechas, como es **dateparser**:

```
pip3 install dateparser
```

Usando su método `parse` podemos convertir un texto a fecha, obteniendo un *datetime*. También funciona con fechas escritas en diferentes idiomas:

```
>>> from dateparser import parse
>>> fecha = '18 de mayo de 2004'
>>> parse(fecha)
datetime.datetime(2004, 5, 18, 0, 0)
```

## Instalación de nuevo módulos

Si bien es cierto que son muchos los módulos que incluye la librería estándar, en el mundo Python tenemos la suerte de que miles de programadores están constantemente desarrollando, mejorando y creando nuevos módulos que a veces sólo facilitan algunas tareas mientras que otras abren nuevos campos de aplicación.

Por eso es importante tener un sistema de administración y gestión de módulos para poder instalar y actualizar los paquetes instalados.

Por defecto en todas las instalaciones de Python, a partir de la versión 3.4, se incluye la herramienta *pip* (*Pip installs packages*, ¡sí el nombre es recursivo!) que nos facilita la tarea de instalar paquetes que no forman parte de la librería estándar.

Y ¿dónde podemos encontrar estos módulos? El mejor repositorio de paquetes es [Python Package Index](https://pypi.org/) (PyPI)



PyPi Logo

El proceso de instalación de un paquete con pip tiene varias etapas:

- Se localiza el paquete en el repositorio de pip.
- Se estudian sus dependencias (los paquetes de los que depende y las versiones necesarias para que funcione).
- Se comprueba si los paquetes de los que depende están instalados, y en las versiones correspondientes.
- Se instalan aquellos que nos faltan.
- Si se cumplen todas las dependencias se instala el paquete
- La instalación de los paquetes hace que estos estén disponibles desde nuestro código

A veces para la instalación de un paquete python es necesario que tengamos instalada una librería o un software previamente. Algunos paquetes están configurados para descargar el código fuente de estas librerías y compilarlo si no las tenemos instaladas previamente.

## Uso de pip

Si queremos instalar un módulo con pip haremos desde la consola de nuestro sistema operativo (no desde dentro del intérprete Python)

```
$ pip install paquete
```

La herramienta *pip* buscará en el repositorio *PyPi* ese paquete, detectará sus dependencias e instalará todos los módulos necesarios para que “paquete” funcione.

Si usamos Google Colab usaremos el comando “!pip” para las instalaciones, y dado que los ficheros se borrarán al desconectar, tendremos que incluir unas instrucciones al principio de nuestro documento para que se instale todo lo necesario antes de que podamos utilizarlo.

### Install the required libs

```
[2] #@title Install the required libs
!pip install -qq git+https://github.com/huggingface/diffusers.git
!pip install -qq "ipywidgets>=7,<8"
!pip install -qq bitsandbytes

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing wheel metadata ... done
```

Cuando instalamos en Linux con pip podemos hacerlo como usuario o como administrador, el resultado es que se instalará en nuestro directorio personal o en el

sistema. Si instalas como administrador corremos el riesgo de alterar el funcionamiento del sistema.

Al usar pip, se comprueba si existe una nueva versión disponible y en ese caso aparece un mensaje que nos recomienda actualizarlo con el comando

```
$ python3 -m pip install --upgrade pip
```

Si por alguna razón tenemos instalado Python 2 en nuestro sistema podemos asegurarnos de que se instala el paquete para la versión 3 usando *pip3* en lugar de pip.

Vamos a ver más opciones de pip:

- Podemos saber la versión de *pip* que tenemos instalada con:  

```
$ pip --version
```
- Como hemos dicho antes, para instalar un paquete haremos:  

```
$ pip install PackageName
```
- Podemos elegir que se instale una versión concreta de un paquete con:  

```
$ pip install 'PackageName==1.4'
```
- Para actualizar un paquete haremos:  

```
$ pip install --upgrade PackageName
```
- Podemos desinstalar un paquete con:  

```
$ pip uninstall PackageName
```
- Para mostrar los módulos que tenemos instalados usaremos:  

```
$ pip list
```
- Podemos ver los detalles de un módulo que tenemos instalado con:  

```
$ pip show requests
```

Entre los detalles aparecen los módulos que se requieren para usar el módulo y los módulos que requieren de éste.

Hagamos un ejemplo. Voy a instalar el módulo *requests* que usaremos más adelante y que nos permite acceder desde nuestro código a páginas webs:

```
$ pip3 install requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in
/home/javacasm/.local/lib/python3.6/site-packages (2.22.0)
Collecting idna<2.9,>=2.5
 Using cached idna-2.8-py2.py3-none-any.whl (58 kB)
```



```

Requirement already satisfied: chardet<3.1.0,>=3.0.2 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests) (3.0.4)
Requirement already satisfied: certifi>=2017.4.17 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests)
(2019.11.28)
Requirement already satisfied: urllib3!=1.25.0,!=1.25.1,<1.26,>=1.21.1 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests) (1.25.8)
Installing collected packages: idna
 Attempting uninstall: idna
 Found existing installation: idna 2.9
 Uninstalling idna-2.9:
 Successfully uninstalled idna-2.9
Successfully installed idna-2.8

```

- En la primera línea nos dice que al ser un usuario normal no puede instalar en el sistema y se hará la instalación en el directorio del usuario.
- Vemos las dependencias y cómo algunas de ellas ya están instaladas.
- En alguna de las dependencias se muestra que la versión debe cumplir algunas condiciones: urllib3!=1.25.0 ó chardet<3.1.0,>=3.0.2
- Podemos generar un listado de todos los módulos que necesitamos para ejecutar algo y guardarlo en un fichero:

```
$ pip freeze > requisitos.txt
```

En el fichero quedarán los paquetes instalados y las versiones instaladas.

Si queremos reproducir esa configuración de instalación lo haremos con:

```
$ pip install -r requisitos.txt
```

Esto es algo que se hace en entornos profesionales para reproducir la configuración de una máquina en otra.